

Algoritmi e Strutture Dati – Parte A – 22/06/26

A1 – Complessità – Punti ≥ 8

Trovare limiti superiori e inferiori per la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & n < 49 \\ 2T(\lfloor n/4 \rfloor) + 3T(\lfloor n/9 \rfloor) + 7T(\lfloor n/49 \rfloor) + n & n \geq 49 \end{cases}$$

A2 – Nodi valore-bilanciati – Punti ≥ 10

Sia T un albero binario in cui a ogni nodo t è associato un valore intero $t.value$.

Un nodo t di T si dice *valore-bilanciato* se la somma dei valori contenuti nel sottoalbero sinistro di t è uguale alla somma dei valori contenuti nel sottoalbero destro di t . Un sottoalbero vuoto ha somma pari a 0.

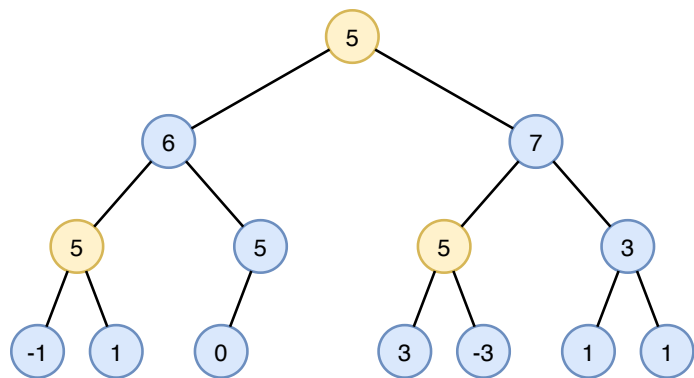
Scrivere un algoritmo

```
int valueBalanced(TREE T)
```

che, dato l'albero binario T , restituisca il numero di nodi valore-bilanciati presenti in T .

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Nell'albero mostrato in figura sono presenti 11 nodi valore-bilanciati, evidenziati in azzurro; pertanto, per tale input, l'algoritmo deve restituire il valore 11.



A3 – Nodi aciclici – Punti ≥ 12

In un grafo orientato $G = (V, E)$, un nodo si dice *aciclico* se non fa parte di nessun ciclo.

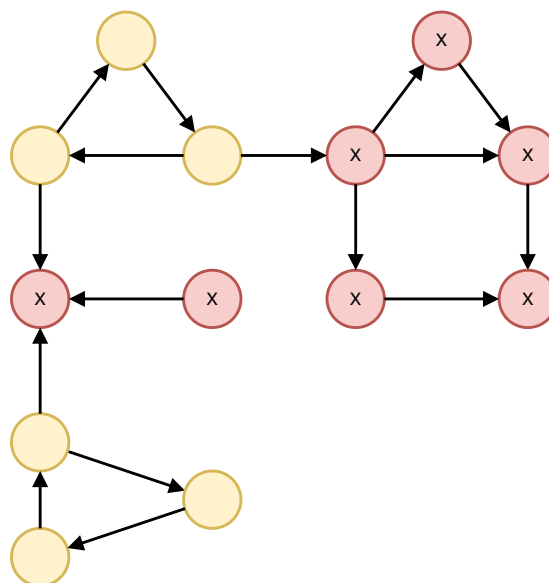
Scrivere un algoritmo

```
int acyclicNodes(GRAPH G)
```

che prenda in input un grafo G e restituisca il numero di nodi aciclici contenuto in esso.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

Nel grafo seguente, i nodi aciclici sono colorati in rosa e contengono una x, quindi l'algoritmo deve restituire 7.



Algoritmi e Strutture Dati – Parte B – 22/06/26

B1 – Shakira – Punti ≥ 8

Dopo il concerto del 14 giugno 2026 all'Intuit Dome di Los Angeles, Shakira deve scegliere l'ordine di esecuzione di n canzoni per il prossimo concerto. Ogni canzone appartiene a uno di quattro tipi: (B) ballad, (D) dance, (L) latino pop, (R) rock.

Vogliamo generare tutte le possibili scalette (liste di canzoni) che rispettano questi vincoli¹

- ogni canzone deve essere eseguita una sola volta;
- non possono esserci tre canzoni consecutive della stessa categoria;

Scrivere un algoritmo

```
setlists(item[] type, int n, int m)
```

che prenda in input un vettore `type[0..n-1]` contenente i tipi delle n canzoni disponibili, la dimensione n del vettore e un intero $0 \leq m \leq n$, e stampi tutte le possibili scalette di lunghezza m , ottenute selezionando m canzoni distinte tra le n disponibili, che rispettano i vincoli sopra indicati.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

B2 – Corsi di laurea – Punti ≥ 10

Sia $CDLs = \{cdl_1, \dots, cdl_k\}$ l'insieme dei k Corsi di Laurea offerti da un'università.

L'università eroga un insieme $C = \{c_1, \dots, c_m\}$ contenente m corsi.

Per ogni Corso di Laurea $CdL_i \in CDLs$, il piano didattico del corso di laurea è descritto da un sottoinsieme $courses_i \subseteq C$ di corsi, di dimensione $m_i = |courses_i|$. Assumiamo che i piani didattici siano disgiunti, cioè $courses_i \cap courses_j = \emptyset$ per ogni $i \neq j$, e dunque ogni corso $c \in C$ appartiene a un solo corso di laurea.

Sono presenti n professori $p_1, \dots, p_n$. Ogni professore p_j ha indicato un sottoinsieme $choices_j \subseteq C$ dei corsi che è in grado di insegnare in modo *competente*. Un professore può comunque insegnare anche corsi non presenti in $choices_j$, ma in tal caso l'insegnamento è considerato *incompetente*. Ogni professore può insegnare al massimo 3 corsi.

Per ogni corso di laurea $CdL_i \in CDLs$ è dato un parametro intero t_i , che rappresenta il numero minimo

di corsi in $courses_i$ che devono essere insegnati da professori competenti.

Descrivere un algoritmo che consenta di verificare se esiste un'assegnazione dei professori ai corsi tale che:

- ogni corso $c \in C$ sia insegnato da esattamente un professore;
- per ogni corso di laurea CdL_i , almeno t_i corsi in $courses_i$ siano assegnati a professori per cui $c \in choices_j$.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

B3 – Hateville PNRR – Punti ≥ 12

Come ricordate, Hateville è un villaggio lineare composto da n case. Il comune di Hateville ha ottenuto un finanziamento PNRR per ridipingere le facciate di tutte le case.

La ditta incaricata dei lavori ha presentato un preventivo sotto forma di una matrice $cost$ di dimensione $n \times k$, dove $k \geq 2$ è il numero di colori disponibili. Per ogni casa h e per ogni colore c , il valore $cost[h][c]$ rappresenta il costo per ristrutturare la facciata della casa h e dipingerla con il colore c .

A Hateville i vicini si odiano: per questo motivo, due case adiacenti devono essere dipinte con colori diversi. L'obiettivo del comune è minimizzare il costo totale della ristrutturazione, scegliendo un colore per ciascuna casa e rispettando il vincolo sui colori delle case adiacenti.

Scrivere un algoritmo

```
int hateville(int[][] cost, int n, int k)
```

che, dato il preventivo $cost$, restituisca il costo minimo totale con cui è possibile ristrutturare tutte le n case.

Discutere brevemente la correttezza dell'algoritmo e la sua complessità computazionale.

¹Avrei potuto aggiungere che "Hips don't lie" and "Waka Waka" non possono essere consecutive; ma aggiunge tanto codice per nulla